RESEARCH PORTFOLIO / STUDY GUIDE

FTQC Resource Estimator

Hardware-aware architecture tradeoffs | Study and interview defense

THE QUESTION

For a fixed logical workload and an explicit reliability budget, how do code distance, decoder assumptions, resource delivery, control latency and platform constraints alter modeled FTQC architecture tradeoffs?

EVIDENCE STATUS

Prototype ; 75 local tests passed, two real-backend tests skipped in packaged validation. Run real Stim/PyMatching before claiming simulated memory results.

HOW TO USE THIS GUIDE

- Read the actual source modules while studying the short theory explanations.
- Re-run the repository tests and examples before defending their claims.
- Answer the worked reasoning and supervisor questions without looking at solutions.
- Document what you independently validated. Tests passing does not automatically make a scientifically calibrated study.

This guide describes the repository as packaged on 25 October 2026. New claims require new reproducible evidence. No source code is copied from referenced research repositories.

01 / Repository code map

Start with these paths in the repository. Each description refers to implemented code, not an aspirational feature.

model.py + reliability.py

Heuristic logical-failure baseline, Wilson intervals, cumulative error and exact repetition-code checks.

architecture.py + study.py

Workload/config parsing, site-equivalent accounting, resource throughput, latency bottlenecks, Pareto and sensitivity studies.

stim_memory.py + memory_analysis.py

Specified rotated surface-code MEMORY simulations and strict matching of measurement metadata to memory-only resource accounting.

hardware_profiles.py + hardware_study.py

Independent photonic, superconducting and trapped-ion toy scheduling/cost profiles. Not calibrated hardware forecasts.

queue_simulation.py + sample_planning.py

Independent queue sanity check; shot-budget calculations for zero-observation uncertainty.

tests/ + scripts/verify_scientific_release.py

Fail-closed checks, protocol replay and integrity gate; external Stim validation remains owner action.

Trace at least one call path from input configuration through model, output file, and unit test. Your goal is to explain why every computation exists, not simply point to its file.

02 / Concepts you must understand

Reliability depends on experiment identity

A logical-error estimate has meaning only with code, circuit, noise model, syndrome rounds, decoder and observable fixed. The optional Stim protocol is a 9-round rotated surface-code X-memory event, not a logical gate, GKP memory, or architecture-level computation.

Statistical evidence

Monte Carlo p-hat = failures / shots; Wilson confidence bounds quantify binomial sampling uncertainty. For zero observed failures, the upper bound remains positive. A 95% interval for one setting is not a simultaneous 95% guarantee for nine conditions.

Cost model and units

A schematic rotated patch needs d^2 data sites plus d^2-1 syndrome sites: 2d^2-1 site equivalents. Additional factory capacity, feedforward or decoding time enters assumed resource/latency formulas; these are not measured physical costs.

Throughput and Pareto reasoning

Idealized resource supply time depends on required accepted states / (parallel source rate × successful delivery probability). Modeled elapsed time is the maximum of supply and processing paths when perfect overlap is assumed. A Pareto point is nondominated only with respect to specified comparable objectives and feasibility criteria.

Hardware adapters are hypotheses

Photonic inputs concern transmission, heralding and feedforward; superconducting inputs concern gate/readout schedule; ion inputs concern zones and movement. Their native cost units differ; cross-platform rankings would be ungrounded.

03 / Worked reasoning

Do these with paper and pencil first. They are illustrative exercises, not claims about measured hardware.

Worked problem 1

For one logical patch at distance d=3, the simple memory patch proxy is 3^2 data + (3^2-1) checks = 17 sites; at d=5 it becomes 25+24=49. The factor 49/17 is an approximate data-plus-check site ratio, not a physical-qubit-to-photon ratio.

Worked problem 2

For four identical independent memory events each with event failure probability q, total modeled failure probability is 1-(1-q)^4. Derive it from the probability that all four survive, then explain why correlated errors invalidate this shortcut.

Worked problem 3

For hypothetical photonic herald success h=0.8 and two path segments of transmissions 0.9 and 0.95, successful delivery probability is 0.8×0.9×0.95=0.684. If the model requests 684 accepted resources, expected raw attempts are 684/0.684=1000. This says nothing about GKP logical fidelity.

One-minute synthesis: state the model, write its central assumption, give one consequence, and name one condition under which that conclusion would fail.

04 / Guided source walkthrough

1. Open examples/architecture_scenarios.json and identify the common workload and which knobs are allowed to change. Write down units.
2. Trace study.load_configuration -> architecture.evaluate -> study.report. Identify the exact place where heuristic logical error enters and where a robust Wilson bound is applied.
3. Read hardware_profiles.PhotonicScenario.evaluate. Manually compute the delivery probability and locate the boundary where supply_us = measurement_us.
4. Inspect memory_analysis.validate_stim_manifest: list metadata fields that must match and the rejected category errors. Explain why mismatched records must not be averaged.
5. Read scripts/verify_scientific_release.py and explain why same-version replay improves auditability without proving physical validity.
6. Run the owner checklist; do not confuse synthetic unit-test fixtures with actual Stim samples.

05 / Re-run experiments and tests

Run the following from this repository root with a compatible Python interpreter. Some commands require third-party dependencies, as explicitly noted.

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
bash RUN_HARDWARE_STUDY.sh
PYTHONPATH=src python3 -m ftqc_resource_estimator.sample_planning --logical-qubits 4 --memory-windows 10
--target-failure 0.001 --shots 20000
bash RUN_STIM_MEMORY.sh # requires installed stim, pymatching, numpy
bash RUN_ALL_VALIDATIONS.sh # must fail unless actual real-backend run succeeds
```

What to inspect afterwards

- outputs/hardware_profiles/HARDWARE_STUDY.md: within-platform scenarios, not real-hardware rankings.
- outputs/stim_memory/observations.json: absent until executed; read versions, noise knobs, seed and actual counts.
- outputs/stim_memory/analysis/MEMORY_REPORT.md and memory_tradeoffs.csv: conditional modeled memory comparisons, not gate error estimates.
- outputs/stim_memory/VALIDATED_RUN.json: integrity/replay evidence after successful execution, not physical model calibration.

06 / Exercises and evidence deliverables

Analytical

Derive d=3 and d=5 patch counts, cumulative failure from independent event assumptions, and why Wilson upper95 can prevent declaring a design feasible.

Deliverable: a brief derivation, reproducible numerical table or short code/experiment note, with conditions and caveats.

Reproducibility

Complete OWNER_NEXT_STEPS.md P0–P1; verify nine conditions at 20,000 shots each; describe any violations or uncertainties.

Deliverable: a brief derivation, reproducible numerical table or short code/experiment note, with conditions and caveats.

Architecture

Change one photonic path transmission and one factory count separately; determine the modeled bottleneck crossover and its sensitivity.

Deliverable: a brief derivation, reproducible numerical table or short code/experiment note, with conditions and caveats.

Scientific critique

Identify two missing noise channels and one hidden timing dependency for every hardware adapter; propose how to measure them.

Deliverable: a brief derivation, reproducible numerical table or short code/experiment note, with conditions and caveats.

07 / Scientific integrity checklist

- Distinguish a theoretical identity, deterministic model result, Monte Carlo estimate, and actual hardware measurement.
- Keep control parameters, physical units, solver/decoder identity, sample count and version provenance visible.
- Compare configurations using a matched workload; do not equate unlike code families or error models.
- Explain one plausible negative result and what you would change in the research design.
- Credit the relevant literature and personally verify methods, code, and results.

08 / Supervisor-style oral examination

Practice answering these questions in 60–120 seconds each. The italic-like answer cues below are study rubrics, not scripts to memorize.

1. What is your most defensible result?

Answer should include: A repeatable *conditional sensitivity analysis* and, only after it actually runs, a specified surface-code memory simulation. Neither is hardware prediction.

2. Why is the same logical error rate reused across multiple cost scenarios?

Answer should include: It isolates assumed architectural costs but ignores correlations between physical timing and error; this is a threat to validity.

3. Would a lower patch count prove architecture A is better?

Answer should include: No. Define task, timing, state preparation, ancilla, calibration, decoding, failure budget, site-time and uncertainty.

4. Is the model specifically for Xanadu?

Answer should include: No; the memory experiment and mathematical framework are hardware-neutral. The photonic adapter is a toy sensitivity hypothesis, without GKP/optical calibration.

5. What is left for you personally?

Answer should include: Execute and audit genuine Stim/PyMatching results, explain intervals and limits, and produce an owner-reviewed research report.

09 / One-hour defense drill

- 15 min — derive one core equation or result and explain an edge case.
- 15 min — open source and trace one example end-to-end, including the unit test.
- 15 min — present one graph or table, uncertainties and alternative explanations.
- 15 min — take skeptical follow-ups about validity, limitations and next experiment.

10 / Reading companion and provenance

These references explain underlying concepts or document relevant public interfaces; they are not repositories whose code was incorporated into this implementation. Only claim a paper reproduction if you have independently reproduced a named result under matched conditions.

Nielsen and Chuang, Quantum Computation and Quantum Information (Ch. 10)

https://www.cambridge.org/9781107002173

Gottesman, Introduction to Quantum Error Correction and Fault-Tolerant Quantum Computation

https://arxiv.org/abs/0904.2557

Stim project documentation

https://github.com/quantumlib/Stim

PyMatching documentation

https://pymatching.readthedocs.io/

Fowler et al., Surface Codes

https://doi.org/10.1103/PhysRevA.86.032324

Bourassa et al., Photonic FTQC Blueprint

https://doi.org/10.22331/q-2021-02-04-392

Your final readiness check

- I can explain the model and its units without consulting the README.
- I can show one derivation and locate where it appears in code.
- I can rerun the experiments, read the actual output and explain sampling uncertainty.
- I can identify at least three limitations and propose a specific falsifiable extension.
- I can state exactly which outcomes have not yet been independently validated.

Read with: README.md, docs/DEVELOPMENT_HISTORY.md, docs/RESEARCH_INTEGRITY.md, and the source/tests named in Section 01. For the resource estimator, also complete OWNER_NEXT_STEPS.md.